# Taller práctico: análisis de datos con Pandas

## 1. Introducción

En este taller construirás, paso a paso, un programa de consola que lee datos de ventas y clientes, los limpia, los une y los analiza. Al final tendrás un programa con menú, parecido a una aplicación real.

Vas a repasar a leer archivos CSV con pandas, explorar la estructura de los datos, limpiar datos con errores, validar y unir dos tablas con merge, agrupar y resumir resultados, y hacer tu primer gráfico.

Requisitos: Python 3.10 o superior instalado y conocimientos básicos de variables, listas y funciones.

Archivos que usarás: cliente.csv y ventas.csv, que vienen en la carpeta Data/raw del proyecto.

Entrega: enlace a tu repositorio, enviado de forma individual por medio de la carpeta de la semana 16. Cada entregable pide pantallazos. Un pantallazo debe mostrar la terminal con el comando ejecutado y su resultado completo.

## 2. Antes de empezar: ambiente virtual (Paso 0)

Un ambiente virtual es una carpeta que guarda las librerías de este proyecto por separado de las demás. Así evitas conflictos entre proyectos y todos usan las mismas versiones.

Abre una terminal dentro de la carpeta del proyecto y sigue estos pasos.

Crear el ambiente:

```
# Mac / Linux
python3 -m venv venv

# Windows
python -m venv venv
```

Activar el ambiente:

```
# Windows (PowerShell)
venv\Scripts\Activate.ps1

# Mac / Linux
source venv/bin/activate
```

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Sabrás que el ambiente está activo porque aparece (venv) al inicio de la línea de la terminal. Si no aparece, no continúes: revisa el comando de activación. Recuerda subir el ambiente virtual completo.

Instalar las librerías:

```
pip install -r requirements.txt
```

Ejecutar el programa:

```
python main.py
```

Para salir del ambiente, escribe deactivate.

**Entregable 0: un pantallazo que muestre (venv) al inicio de la línea y el final de la instalación de las librerías.**

## 3. Mapa del proyecto

Antes de programar, revisa cómo está organizado el proyecto. Cada archivo cumple una función, y el menú de main.py llama a cada módulo según la opción que elijas.

```
proyecto/
├── main.py
├── carga.py
├── limpieza.py
├── merge.py
├── analisis.py
├── graficos.py
├── exploracion.py
├── requirements.txt
└── Data/
    ├── raw/
    │   ├── cliente.csv
    │   └── ventas.csv
    └── processed/
```

- main.py: muestra el menú y llama a los demás módulos según la opción elegida. No lo modifiques.
- carga.py: lee los archivos CSV. Ya está completo.
- exploracion.py: ejemplos de indexación, .loc, .iloc y concat. Se ejecuta por separado, no desde el menú.
- limpieza.py: limpia clientes y ventas. Debes completarlo.
- merge.py: valida y une las dos tablas. Debes completarlo.
- analisis.py: responde las preguntas del desafío. Debes completarlo.
- graficos.py: dibuja dos gráficos sencillos. Debes completarlo.
- Data/raw: los archivos originales. No los modifiques; el programa siempre carga los datos desde aquí.
- Data/processed: resultados que genera el programa, como merge.csv y los gráficos.

El programa busca los archivos a partir de la ubicación del propio módulo, así que debes ejecutar python main.py desde la carpeta del proyecto.

# 4. Parte 1: lectura y exploración

## 4.1 Leer los archivos

La función cargar_datos() de carga.py usa pd.read_csv() para leer cada archivo y convertirlo en una tabla llamada DataFrame. Ya está escrita; por ahora solo úsala desde la opción 1 del menú.

## 4.2 Leer solo algunas columnas

El parámetro usecols indica qué columnas leer. Es útil cuando un archivo es muy grande y solo necesitas unas pocas. Prueba esta línea en un archivo de prueba, dentro de la carpeta del proyecto:

```
ventas_reducido = pd.read_csv("Data/raw/ventas.csv", usecols=["producto", "total"])
```

La lista de columnas va entre corchetes, y las columnas se escriben tal como aparecen en el archivo.

## 4.3 Explorar la estructura

Para trabajar con datos reales primero hay que saber qué tienes. Las siguientes herramientas dan una primera visión. Como vas a ejecutar un archivo .py, las líneas sueltas no muestran nada; por eso usamos print() cuando queremos ver un resultado.

.shape: indica cuántas filas y columnas tiene la tabla. Para ventas.csv, antes de limpiar, debe mostrar (68, 10).

```
print(ventas.shape)
```

.info(): muestra los nombres de las columnas, cuántos valores no nulos tiene cada una y su tipo de dato. Si una columna tiene menos valores no nulos que el total de filas, tiene datos faltantes. Revisa la columna producto de ventas: ¿cuántos valores no nulos tiene?

```
ventas.info()
```

.describe(): calcula estadísticas básicas (cantidad, promedio, mínimo, máximo y cuartiles) de las columnas numéricas. Ignora las columnas de texto.

```
print(ventas.describe())
```

**Entregable 1: pantallazos de .shape, .info() y .describe() de ventas.csv y de clientes.csv, y la salida completa de exploracion.py.**

## 4.4 Acceder a las columnas

Puedes acceder a una columna con notación de punto o con corchetes. Ambas dan el mismo resultado, pero los corchetes son más seguros: funcionan con nombres que tienen espacios y no chocan con los métodos de pandas. Por ejemplo, ventas.count es un método, no una columna.

```
print(ventas.total)
print(ventas["total"])
```

## 4.5 Indexación

Cada fila de una tabla tiene una etiqueta, llamada índice. Por defecto, pandas numera las filas desde 0. Si una columna identifica cada fila (en ventas es id_venta), puedes usarla como índice con set_index(). Entonces la columna pasa a ser la etiqueta de cada fila.

```
ventas = ventas.set_index("id_venta")
```

Usamos set_index() en lugar de asignar directamente ventas.index = ventas.id_venta porque deja explícito qué columna se usa como índice.

Con un índice de texto puedes buscar filas por su etiqueta. Eso se hace con .loc y .iloc, que se explican a continuación.

### 4.6 .loc y .iloc

Hay dos formas de seleccionar datos de una tabla:

- .loc busca por etiqueta: le indicas el nombre de la fila (en este caso, el ID de venta) y el nombre de la columna.
- .iloc busca por posición: le indicas el número de fila y de columna, empezando en 0.

Hay una diferencia clave en los rangos. Con .loc el final del rango sí se incluye; con .iloc no. Por eso .loc['V001':'V006'] devuelve seis filas, mientras que .iloc[0:5] devuelve solo cinco.

Ejecuta exploracion.py desde la terminal con python exploracion.py. Este archivo no se llama desde el menú.

```
print(ventas.loc["V003", "total"])
print(ventas.iloc[2, 6])
print(list(ventas.loc["V001":"V006"].index))
print(list(ventas.iloc[0:5].index))
```

Tu tarea: ejecuta exploracion.py y responde en tu entregable por qué .iloc[2, 6] devuelve el total de la venta V003. Pista: después de set_index, id_venta ya no es una columna. Cuenta las columnas empezando en 0.

### 4.7 Unir partes de una tabla con concat()

pd.concat() junta varias tablas, una debajo de otra. En exploracion.py separamos las ventas de abril y de mayo y luego las volvemos a unir. El resultado debe tener las mismas filas que la tabla original.

```
total = pd.concat([parte_1, parte_2], ignore_index=True)
```

## 5. Parte 2: limpieza de datos (limpieza.py)

Los datos reales casi nunca llegan listos para usarse. Antes de analizar hay que detectar y corregir los problemas. Este paso se llama limpieza, y el menú lo ejecuta desde la opción 2.

Los problemas más comunes son los valores nulos (celdas vacías), las filas duplicadas, los espacios sobrantes en los textos y las fechas escritas como texto. Las herramientas de la Parte 1 y las que verás aquí te permiten encontrarlos. No hay una lista de errores: parte del trabajo es encontrarlos tú.

En estos archivos hay pocos errores. Si encuentras menos problemas de los que esperabas, está bien: el objetivo es aprender a buscarlos, no encontrar un número fijo.

## 5.1 Valores nulos

Para contar los nulos de cada columna usa:

```
print(ventas.isnull().sum())
```

Tienes dos formas de tratarlos. Con .dropna() eliminas las filas que tengan nulos, pero puedes perder información valiosa: una venta sin nombre de producto sigue teniendo un total real. Con .fillna() reemplazas el nulo por un valor, por ejemplo:

```
datos["nota"] = datos["nota"].fillna(0)
```

El valor de reemplazo depende del dato. Para un nombre de producto, un texto como 'Sin especificar' tiene sentido. Para una edad o un precio, el valor debe tener una justificación. En ventas.csv hay un producto vacío: trátalo con fillna y explica en tu entregable por qué no usaste dropna.

## 5.2 Duplicados

Una fila duplicada es una fila idéntica a otra. Para contarlas usa .duplicated().sum(). Para eliminarlas usa .drop_duplicates(). Después de eliminar filas, el índice queda con huecos; .reset_index(drop=True) lo vuelve a numerar desde 0.

```
print(ventas.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
```

## 5.3 Limpieza de texto

Los textos pueden llegar con espacios sobrantes al inicio o al final, o con mayúsculas inconsistentes. Por ejemplo:

```
nombres = pd.Series(["  ana ", "luis  "])
print(nombres.str.strip().str.title())
```

Con .str.strip() quitas los espacios sobrantes. Con .str.title() pones en mayúscula la inicial de cada palabra. Cuando trabajas con columnas de texto, el .str es obligatorio.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Revisa también si hay textos que deberían ser iguales pero están escritos de forma diferente. Esto no siempre se ve a simple vista; .value_counts() te ayuda a encontrarlo.

### 5.4 Tipos de datos y fechas

Una fecha escrita como texto no se puede ordenar ni restar. Para convertirla en fecha usa pd.to_datetime() e indica en qué formato viene:

```
pd.to_datetime("15/03/2026", format="%d/%m/%Y")
```

En el formato, %Y es el año de cuatro dígitos, %m el mes y %d el día. Con errors='coerce', una fecha que no se pueda convertir se reemplaza por un valor vacío en lugar de detener el programa.

Antes de escribir el código revisa el formato de fecha_venta en ventas.csv. Después de limpiar, comprueba con .info() que la columna tenga tipo datetime.

### 5.5 Entregable 2

**Pantallazos de la opción 2 del menú (con los nulos y duplicados antes y después de la limpieza) y del tipo de la columna fecha_venta después de convertirla. Explica en un párrafo corto qué problemas encontraste y cómo los trataste.**

## 6. Parte 3: validación y merge (merge.py)

Muchas veces la información está repartida en varias tablas. En este caso, ventas.csv tiene el id del cliente, pero no su nombre ni su ciudad; esos datos están en cliente.csv. El merge une las dos tablas usando una columna en común: id_cliente.

Piensa en el merge como buscar, para cada venta, el cliente que le corresponde. Si una venta tiene id_cliente 21, el merge trae el nombre, la ciudad y el segmento del cliente 21.

Para unir dos tablas se usa esta forma general:

```
df = tabla_a.merge(tabla_b, on="columna_comun", how="left")
```

El parámetro how='left' significa que se conservan todas las filas de la tabla de la izquierda (ventas), aunque no encuentren pareja en la otra. El parámetro validate='m:1' revisa que varias ventas puedan apuntar a un mismo cliente, pero que cada cliente aparezca una sola vez en la tabla de clientes. Si un cliente estuviera repetido, el merge duplicaría ventas sin avisar, y validate lo detecta.

Antes de unir hay que validar: que exista la columna id_cliente en ambas tablas, que no haya id_cliente repetidos en clientes, que el tipo de dato coincida, y cuántas ventas no tienen cliente. Esa validación está en validar_premerge(), y tu trabajo es completarla (TODO 6).

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

Luego completa unir() (TODO 7). Debe validar, hacer el merge, guardar el resultado en Data/processed/merge.csv y devolver el dataframe.

Revisa la salida. El merge debe tener el mismo número de filas que ventas, y su número de columnas debe ser la suma de las columnas de ambas tablas menos la columna en común. ¿Por qué? Explícalo en tu entregable.

Entregable 3 (parte 1): pantallazo de la opción 3 con la validación y el resultado del merge, y tu explicación del número de columnas.

## 7. Parte 4: análisis y agrupaciones (analisis.py)

Ya tienes una tabla con todos los datos juntos. Ahora debes responder preguntas de negocio. La herramienta principal es groupby(), que sigue tres pasos: separa las filas en grupos según una columna, aplica una operación a cada grupo (por ejemplo, sumar) y junta los resultados en una tabla nueva.

Por ejemplo, para sumar las ventas por categoría:

```
df.groupby("categoria")["total"].sum()
```

### 7.1 agg: varias operaciones a la vez

Cuando necesitas varios resultados por grupo, usa .agg() con nombres claros:

```
resumen = df.groupby("columna").agg(nombre_nuevo=("columna_a", "sum"))
```

Cada línea define una columna nueva. A la izquierda está el nombre que tendrá, y a la derecha, la columna y la operación: sum suma y count cuenta.

### 7.2 value_counts()

Cuenta cuántas veces aparece cada valor de una columna:

```
df["columna"].value_counts()
```

### 7.3 nunique()

nunique() cuenta valores distintos. Sirve, por ejemplo, para contar cuántos clientes diferentes hay en un grupo, aunque cada cliente tenga varias compras:

```
df.groupby("grupo")["id"].nunique()
```

Ojo con las comparaciones: si un grupo tiene más clientes que otro, los totales no son comparables. Para comparar grupos de distinto tamaño, divide el total entre el número de clientes y compara el promedio por cliente.

### 7.4 Preguntas que responde analisis.py

Cada pregunta corresponde a una función de analisis.py. Completa el TODO correspondiente:

- TODO 8, ingresos_por_categoria: ¿qué categoría genera más ingresos? Suma el total por categoría y ordena de mayor a menor.
- TODO 9, canal_que_mas_vende: ¿qué canal vende más? Muestra el número de ventas y los ingresos por canal. En tu respuesta indica qué criterio usaste: cantidad de ventas o ingresos.
- TODO 10, clientes_vip_vs_regular: ¿los clientes VIP compran más que los Regulares? Compara compras por cliente y gasto por cliente, no los totales, porque hay distinta cantidad de clientes en cada segmento.
- TODO 11, metodo_pago_mas_usado: ¿qué método de pago es el más usado? Cuenta las ventas por método de pago.
- TODO 12, top5_clientes: ¿cuáles son los cinco clientes con más compras? Ordena por número de compras y usa el gasto total como desempate. Pista: .head(5) toma las primeras cinco filas.

### 7.5 Entregable 3

**Pantallazos de la opción 4 del menú (la salida completa del análisis) y el código de las funciones que completaste.**

## 8. Parte 5: primer contacto con gráficos (graficos.py)

Esta es tu primera experiencia con gráficos. Los temas se verán con más detalle en la próxima sesión; aquí solo necesitas dibujar barras y guardarlas como imagen.

Un gráfico de barras necesita tres cosas: los datos ya agrupados, la instrucción que lo dibuja y los textos que lo explican. Por ejemplo:

```
datos.plot(kind="bar")
plt.title("Título del gráfico")
plt.savefig("grafico.png", bbox_inches="tight")
plt.show()
```

- datos.plot(kind="bar") dibuja las barras con los datos de una tabla agrupada, como el resultado de groupby.

- plt.title() pone el título. plt.xlabel() y plt.ylabel() ponen el texto de cada eje.
- plt.savefig() guarda la imagen como archivo .png. Debe ejecutarse antes de plt.show().
- plt.show() muestra la ventana del gráfico. Cierra la ventana para que el programa continúe.
-

Tu tarea: completa grafico_ventas_por_categoria() (TODO 13) y grafico_ventas_por_canal() (TODO 14). Sigue los pasos del docstring de cada función.

Al ejecutar la opción 5 del menú se abrirán dos ventanas. Cierra cada una para que el programa siga. Las imágenes se guardan en Data/processed/graficos.

**Entregable 4: los dos archivos .png generados y el código de graficos.py que completaste.**

## 9. Desafío final

Responde las preguntas de negocio usando tu programa. Para cada respuesta debes incluir el resultado copiado de la terminal, una frase con la respuesta y el criterio que usaste.

1. ¿Qué categoría genera más ingresos?
2. ¿Qué canal vende más?
3. ¿Los clientes VIP compran más que los Regulares?
4. ¿Qué método de pago es el más usado?
5. ¿Cuáles son los cinco clientes con más compras?

Requisitos del reto:

- Limpiar ambos datasets con limpieza.py.
- Unir las tablas con merge() en merge.py.
- Usar groupby, loc/iloc, fillna o dropna, y concat o merge. Escribe por tu cuenta al menos una línea con loc o iloc y una con concat en tu código.
- Dibujar dos gráficos obligatorios (los de la Parte 5). Un tercer gráfico de barras de tu elección es opcional.
-

**Entregable final: enlace al repositorio con el código (.py), los pantallazos de las opciones del menú, los gráficos (.png) y un archivo README.md con tu nombre y las respuestas a las cinco preguntas.**

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net

## 10. Bonus (nivel pro)

Calcula las ventas por mes. Como fecha_venta ya es una fecha, puedes crear una columna con el mes:

```
ventas["mes"] = ventas["fecha_venta"].dt.to_period("M")
ventas.groupby("mes")["total"].sum()
```

Se usa to_period('M') y no .dt.month porque el número del mes (1 a 12) mezclaría abril de 2026 con abril de cualquier otro año. Con to_period('M') cada mes queda identificado por año y mes.

Entregable bonus: la tabla de ventas por mes y una frase que diga qué mes vendió más.

## 11. Rúbrica

| Entregable | Puntaje |
|---|---|
| 0. Ambiente virtual | 5 |
| 1. Exploración (.shape, .info, .describe, exploracion.py) | 15 |
| 2. Limpieza (antes y después, explicación) | 20 |
| 3. Merge y análisis (validación, opción 3 y 4) | 20 |
| 4. Gráficos (.png y código) | 10 |
| Desafío final (respuestas, código, requisitos) | 30 |
| **Total** | **100** |

Bonus: hasta 10 puntos adicionales.

Criterios generales: un pantallazo sin la terminal o sin el comando no cuenta. Un código que no ejecuta pierde la mitad del puntaje de su entregable. Una respuesta sin el criterio usado pierde la mitad del puntaje de esa pregunta.

**Para tener en cuenta:**

Todos los entregables deben estar alojados en un repositorio de GitHub, respetando la estructura de carpetas definida para el proyecto.

Dentro del repositorio, deben crear una carpeta adicional llamada images, donde se almacenarán las capturas de pantalla como evidencia del desarrollo. Estas deben tener nombres descriptivos que permitan identificar su contenido. Eviten nombres genéricos como pantallazo.png, captura.png o 01.png.

Docente: Diego Giraldo Zapata
dgiraldoza@cesde.net